

OFFICIAL PROBLEM STATEMENT

VibeGuard v3.0 — Pre-Push Security Gate & Dependency Audit Framework

TEAM NAME

TEAM SPARK

INSTITUTION

Sreenidhi Institute of
Science and Technology

CHALLENGE DOMAIN

Cybersecurity / DevSecOps

PROJECT

VibeGuard v3.0

TEAM MEMBERS:

B. Poojitha 24311A6201

Y. Bhavya Sri 24311A6203

A. Ranaveer 24311A6211

M. Gagan Sai 24311A6212

01 The Challenge

Core Problem Statement

Modern software development workflows—especially in open-source, startup, and academic settings—expose repositories to a persistent and growing class of security vulnerabilities that are **invisible at commit time**. Developers unknowingly push API keys, private tokens, database credentials, and other sensitive artifacts directly into version-controlled repositories. Simultaneously, third-party dependency ecosystems harbour thousands of known-vulnerable packages that enter production undetected. The lack of an *automated, lightweight, real-time security gate at the developer's workstation* is the critical gap this project addresses.

13M+

GITHUB REPOS WITH EXPOSED
SECRETS (2024)

79%

OF BREACHES INVOLVE
CREDENTIAL LEAKS

\$4.45M

AVERAGE COST OF A DATA BREACH
(IBM 2023)

02 Dimensions of the Problem

🔑 Credential Leakage

Developers frequently commit environment files, configuration files, and hard-coded secrets (AWS keys, GitHub tokens, database passwords, Stripe API keys) alongside application code. Once pushed to remote repositories, these credentials are indexed by automated scanners and exploited within minutes. Even after deletion, they persist in git history indefinitely.

📦 Supply-Chain Vulnerabilities

Modern applications depend on hundreds of third-party packages (npm, pip, cargo, maven). Each dependency carries its own CVE exposure. Outdated packages—often pinned to specific versions for compatibility—may contain critical vulnerabilities (RCE, SQL injection, path traversal) that are exploited through transitive dependency chains.

Late-Stage Detection

Enterprise CI/CD pipelines run security scans post-push, meaning vulnerable code is already in the shared repository when issues are discovered. Remediation at this stage requires coordinated rollbacks, incident response procedures, and potential security breach notifications—dramatically increasing cost and developer disruption.

Alert Fatigue

Existing security tools like Snyk, Dependabot, and GitGuardian generate excessive false positives: test credentials, example configurations, and documentation snippets flagged as real threats. Developers learn to dismiss alerts, creating a dangerous desensitization to genuine security warnings and causing real vulnerabilities to be ignored.

03 Industry Context & Scale

The DevSecOps Gap: While large enterprises deploy comprehensive security infrastructure (SIEM, DAST, SAST), the majority of development teams—especially in hackathon contexts, startups, and academic institutions—operate without any pre-commit or pre-push security layer. Code moves from developer workstation to cloud repository with zero security validation at the critical first-mile checkpoint.

Security Challenge	Current Industry Approach	Limitation	VibeGuard Approach
Secret Detection	GitGuardian, TruffleHog (post-push)	Post-Commit	Pre-Push Gate
Dependency Auditing	Snyk, Dependabot (cloud-based)	Cloud Required	Local + OSV API
Code Pattern Scanning	SonarQube, Semgrep (CI/CD only)	Pipeline Only	Pre-Push Local
Developer Integration	IDE plugins, SaaS dashboards	Tool Fragmented	Single CLI Tool
False Positive Rate	Industry average: 40–60%	High Noise	Near Zero

04 Gap in Existing Solutions

What is Missing from the Market

No widely-adopted tool provides a **unified, developer-local, pre-push security gate** that simultaneously handles secret scanning, source code vulnerability detection, and dependency CVE auditing in a single binary with zero external service dependencies for core functionality.

Existing solutions fragment security across multiple paid SaaS platforms with complex onboarding, require cloud connectivity, generate overwhelming false positives, and operate exclusively in post-commit workflows— meaning security validation happens only after the vulnerable code has already entered the shared repository.

For student developers, hackathon teams, and early-stage startups, the barrier to adopting enterprise-grade security tooling is prohibitively high—in terms of cost, infrastructure requirements, and operational complexity.

- ▶ No free, single-binary tool for simultaneous secret + dependency + code scanning at push time
- ▶ Existing tools require cloud accounts, API keys, or SaaS subscriptions to function
- ▶ CI/CD-only security scanning allows compromised code to exist in shared repositories for hours before detection
- ▶ High false-positive rates in existing tools cause alert desensitization among development teams
- ▶ No offline-capable, cross-platform security gate suitable for resource-constrained development environments
- ▶ Lack of contextual filtering — tools flag test files, example configs, and documentation as real threats
- ▶ No standardised risk-scoring framework that consolidates findings from multiple scan engines into a single actionable verdict

05 Specific Technical Challenge Addressed

The VibeGuard Mission: Build a production-quality, open-source developer security CLI that installs as a global git hook, scans every pre-push payload in under 3 seconds, blocks dangerous commits with precise contextual analysis, and reports vulnerabilities in a structured, actionable format — with zero false positives and zero external service dependencies for offline environments.

Technical Challenge 1: Speed

Security scans must complete in <3 seconds on repositories with 50+ files to avoid developer friction. This requires a compiled, multi-threaded scanning engine rather than interpreted scripting languages — motivating the dual Go+Rust architecture.

Technical Challenge 2: Accuracy

Pattern-based secret detection produces unacceptable false-positive rates when applied naively. The challenge is building a context-aware filter that distinguishes real credentials from placeholder values, test fixtures, and documentation examples.

Technical Challenge 3: Coverage

Dependency ecosystems (Python pip, Node npm, Rust cargo, Java maven) each use different manifest formats and versioning schemes. A unified parser must handle all major ecosystems and query the Open Source Vulnerabilities (OSV) database in real time.

Technical Challenge 4: Integration

A security tool is only effective if developers actually use it. The solution must install with a single command, integrate invisibly into existing git workflows as a pre-push hook, and require zero configuration to provide immediate value.